

EXP.NO: 8	Banker's Algorithm
DATE:	

AIM:

To implement the Banker's Algorithm for deadlock avoidance in C. The program aims to determine if a system is in a safe state after resource allocation and to find a safe sequence of process execution if one exists.

ALGORITHM:

1. **Preparation:** Calculate the **Need** for each process ($\text{Max} - \text{Allocation}$) and set all processes to "Not Finished."
2. **Selection:** Look for a process that is "Not Finished" and whose **Need** is less than or equal to the currently **Available** resources.
3. **Release:** If you find one, "pretend" it finishes. Add its **current Allocation** back into the **Available** pool and mark it "Finished."
4. **Conclusion:**

If **all** processes finish, the system is **Safe**.

If you get stuck and some are "Not Finished," the system is **Unsafe**.

CODE:

```
#include <stdio.h>

int main() {
    int n, r;
    scanf("%d %d", &n, &r);

    int alloc[n][r], max[n][r], need[n][r], avail[r], finish[n], seq[n];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < r; j++)
            scanf("%d", &alloc[i][j]);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < r; j++) {
```

```
    scanf("%d", &max[i][j]);
    need[i][j] = max[i][j] - alloc[i][j];
}
```

```
for (int j = 0; j < r; j++) scanf("%d", &avail[j]);
```

```
for (int i = 0; i < n; i++) finish[i] = 0;
```

```
int count = 0;
```

```
while (count < n) {
```

```
    int found = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (!finish[i]) {
```

```
            int ok = 1;
```

```
            for (int j = 0; j < r; j++)
```

```
                if (need[i][j] > avail[j]) ok = 0;
```

```
            if (ok) {
```

```
                for (int j = 0; j < r; j++)
```

```
                    avail[j] += alloc[i][j];
```

```
                seq[count++] = i;
```

```
                finish[i] = 1;
```

```
                found = 1;
```

```
            }
```

```
        }
```

```
    }
```

```
    if (!found) break;
```

```
}
```

```
if (count == n) {
```

```
    printf("Safe\n");
```

```
    for (int i = 0; i < n; i++) printf("P%d ", seq[i]);
```

```
} else {
```

```
    printf("Not Safe");
```

```
}
```

```
return 0;
```

```
}
```

OUTPUT:

```
navdeep@localhost:~/os_exp

[navdeep@localhost ~]$ pwd
/home/navdeep
[navdeep@localhost ~]$ ls
a.out  Desktop  Downloads  os      Pictures  Templates
cpu    Documents  Music      os_exp  Public    Videos
[navdeep@localhost ~]$ cd os_exp
[navdeep@localhost os_exp]$ ls
awk      banker.c  fcfs      first_fit.c  priority  RR      shell  sys
awk.txt  best_fit.c  fcfs.c    LRU.c        priority.c  RR.c    SJF    sys.txt
banker   e.txt      FIFO.c    pc.c         receiver.c  sender.c  SJF.c
[navdeep@localhost os_exp]$ gcc banker.c -o banker
[navdeep@localhost os_exp]$ ./banker
5 3
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
3 3 2
Safe
[navdeep@localhost os_exp]$
```

RESULT:

The program successfully takes inputs for n processes and r resources. It computes the Need matrix and simulates the Banker's safety check.